

LIBRARY MANAGEMENT SYSTEM

System Design & Technical Documentation

Prepared By: Prajakta Pare | Trainee Software Engineer |

Table of Contents

No.	Section Title	Pg.
1	Introduction	4
2	Problem Definition	5
2.1	Background	
2.2	Problems with the Existing System	
2.3	Proposed Solution	
3	Need of the System	6
4	Technology Stack	6
5	Entity-Relationship Diagram (ERD)	8
6	API Design	9
6.1	Base URL	
6.2	Authentication	
6.3	Response Format	
6.4	Auth Endpoints	
6.5	User Endpoints	
6.6	Book Endpoints	
6.6	Book Request Endpoints	
6.8	Issue & Return Endpoints	
6.9	Role Endpoints	
7	Flow Diagrams	11
8	System Architecture Diagram	14
9	Validation Strategy	15
10	Future Enhancements	16

1. Introduction

The Library Management System is developed to overcome the limitations of traditional library processes, where maintaining records of books, student details, book requests, and issue/return activities manually becomes time-consuming and difficult to manage. As the number of users and books increases, manual operations can lead to delays, errors, data inconsistency, and reduced efficiency.

This project provides a centralized web-based platform that automates and simplifies library operations. It allows administrators to manage books, users, and requests efficiently, while students can register, search books, request borrowing, and track issued books through an easy-to-use interface. The system improves accuracy, reduces manual effort, and enhances the overall library management process.

2. Problem Definition

2.1 Background

Traditional library management relies heavily on manual processes physical registers for tracking issued books, handwritten student records, and manual fine calculations. This approach is error-prone, time-consuming, and difficult to scale.

2.2 Problems with the Existing System

- No centralized record of book availability; students must physically check shelves.
- Manual tracking of due dates leads to missed returns and uncollected fines.
- Lack of a request/reservation system causes conflicts and inequitable access.
- Administrators spend significant time on repetitive tasks such as issuing books and recording returns.
- No audit trail for issued books, returns, or fine payments.

2.3 Proposed Solution

- A role-based access system distinguishing administrators from students.
- A digital catalog of books with availability tracking.
- A request workflow where students apply for books and administrators approve/issue.
- Automated fine calculation based on due dates and return dates.
- Full audit history for all transactions.

3. Need of the System

The need for the Library Management System arises from the challenges of managing library operations manually, which can lead to inefficiencies, errors, and difficulties in maintaining accurate records. Traditional methods of handling book inventories, student registrations, issue and return transactions, and request approvals become increasingly complex as the number of users and books grows. This system provides a secure and centralized solution by implementing user authentication and role-based access control, ensuring that administrators and students have appropriate permissions. It enables efficient book catalog management with real-time availability tracking, supports a structured request and approval process, automates issue and return records, and calculates fines accurately without manual intervention.

Additionally, email verification enhances security by allowing only valid users to access the system. Overall, the system improves operational efficiency, reduces administrative workload, and delivers a more organized and reliable library experience.

4. Technology Stack

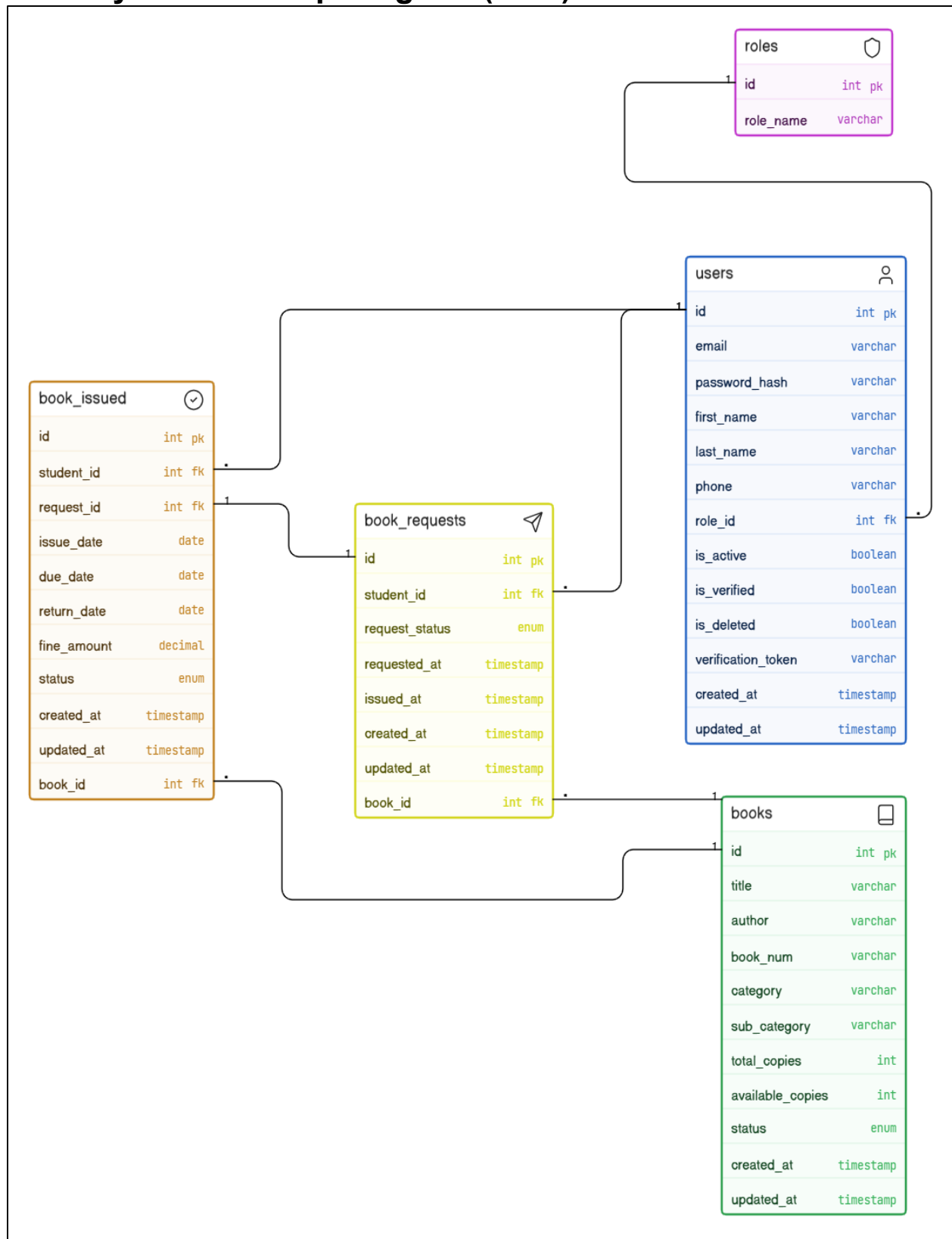
4.1 Core Technology Stack

Category	Technology	Purpose
Environment	Node.js	Executes JavaScript on the server side
Web Framework	Express.js	Handles routing, middleware, and HTTP requests
Database	MySQL	Stores and manages relational data

4.2 NPM Packages and Libraries

Package	Category	Purpose
mysql2	Database Connectivity	MySQL connection pooling and query execution
jsonwebtoken	Authentication	JWT generation and verification
bcrypt	Security	Password hashing and comparison
nodemailer	Communication	Sending emails and notifications
ajv	Validation	JSON Schema-based request validation
winston	Logging	Application logging and error tracking
dotenv	Configuration	Environment variable management
eslint	Code Quality	Static code analysis and linting
prettier	Code Formatting	Automatic code formatting

5. Entity-Relationship Diagram (ERD)



From	To	Relationship	On Key
users	roles	Many-to-One	users.role_id → roles.id
book_requests	users	Many-to-One	book_requests.student_id → users.id
book_requests	books	Many-to-One	book_requests.book_id → books.id
book_issued	book_requests	One-to-One	book_issued.request_id → book_requests.id
book_issued	users	Many-to-One	book_issued.student_id → users.id
book_issued	books	Many-to-One	book_issued.book_id → books.id

6. API Design

6.1 Base URL

`http://localhost:3000`

6.2 Authentication

All protected routes require a JWT token via cookie (`access_token`) set on login. The middleware extracts and verifies the token, attaching `req.user = {id, role_id}`.

6.3 Response Format

```
{"success_flag": true, "message": "SUCCESS_MESSAGE", "data": {}}
{"success_flag": false, "message": "ERROR_MESSAGE"}
```

6.4 Auth Endpoints

Method	Endpoint	Access	Description
POST	/auth/login	Public	Login with email & password
GET	/auth/verify-email?uid=&token=	Public	Verify email via token
POST	/auth/forgot-password	Public	Send password reset email
POST	/auth/reset-password	Public	Reset password with token
POST	/auth/logout	Public	Clear auth cookie

6.5 User Endpoints

Method	Endpoint	Access	Description
GET	/users/profile	Authenticated	Get own profile
PATCH	/users/profile	Authenticated	Update own profile
GET	/users	Admin	Get all users (filterable)
GET	/users/:id	Admin	Get user by ID
POST	/users	Admin	Create new user (sends verification email)
PATCH	/users/:id	Admin	Update user fields
DELETE	/users/:id	Admin	Soft/hard delete user

6.6 Book Endpoints

Method	Endpoint	Access	Description
GET	/books	Authenticated	Get all books (filterable)
GET	/books/:id	Authenticated	Get book by ID
POST	/books	Admin	Create new book
PATCH	/books/:id	Admin	Update book fields
DELETE	/books/:id	Admin	Delete book

6.6 Book Request Endpoints

Method	Endpoint	Access	Description
POST	/requests	Student	Request a book
GET	/requests	Authenticated	Get all requests (filterable) Student can see his own requested books data
GET	/requests/:id	Admin	Get request by ID
PATCH	/requests/:id/approve	Admin	Approve and issue book
PATCH	/requests/:id/reject	Admin	Reject request
DELETE	/requests/:id/cancel	Student	Cancel own pending request

6.8 Issue & Return Endpoints

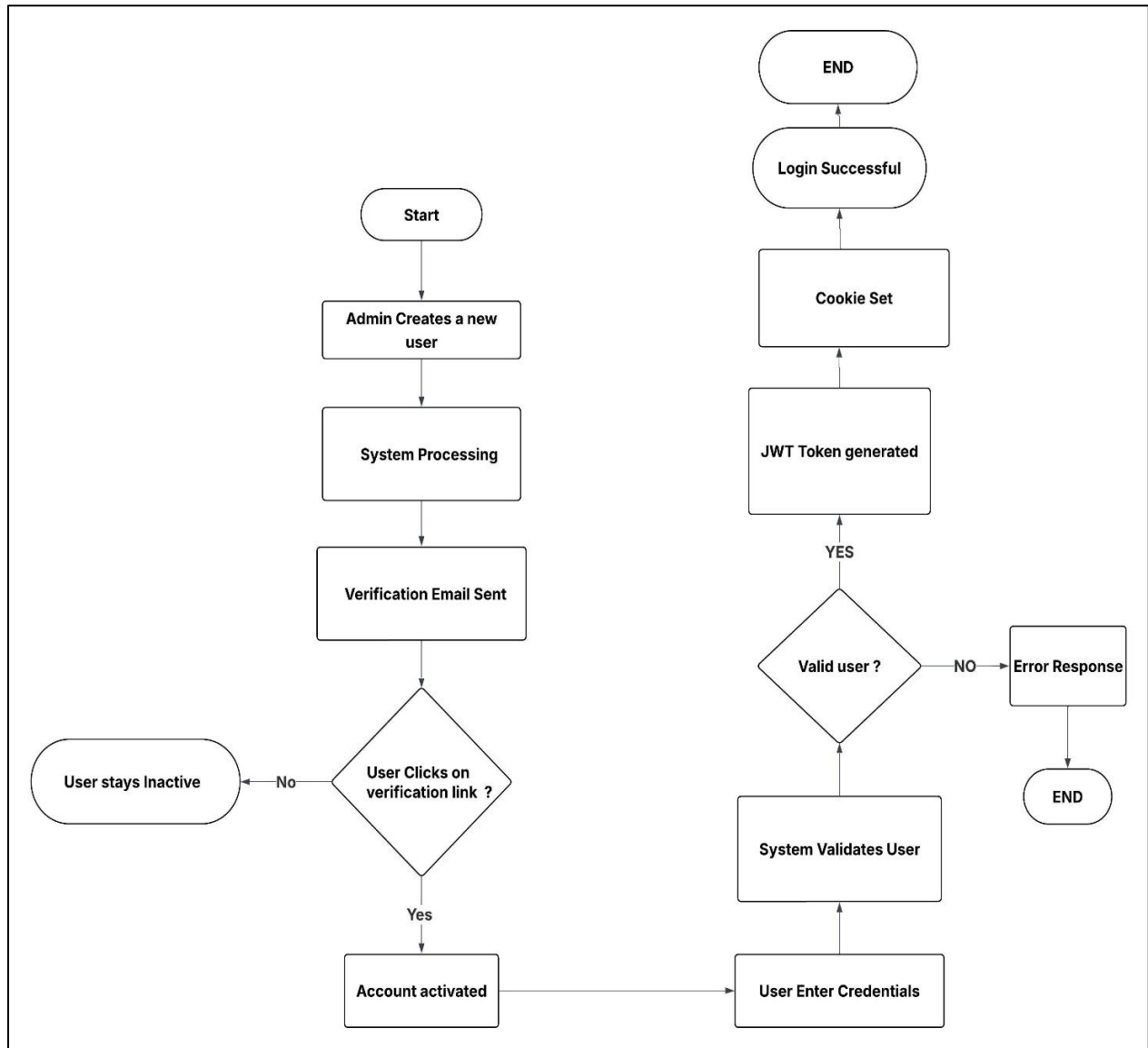
Method	Endpoint	Access	Description
GET	/book-issued	Authenticated	Get all issued books data (filterable) Student can see his own issued or returned books
GET	/book-issued/:issue_id	Admin	Get issue details
PATCH	/book-issued/:issue_id/return	Admin	Return book & calculate fine

6.9 Role Endpoints

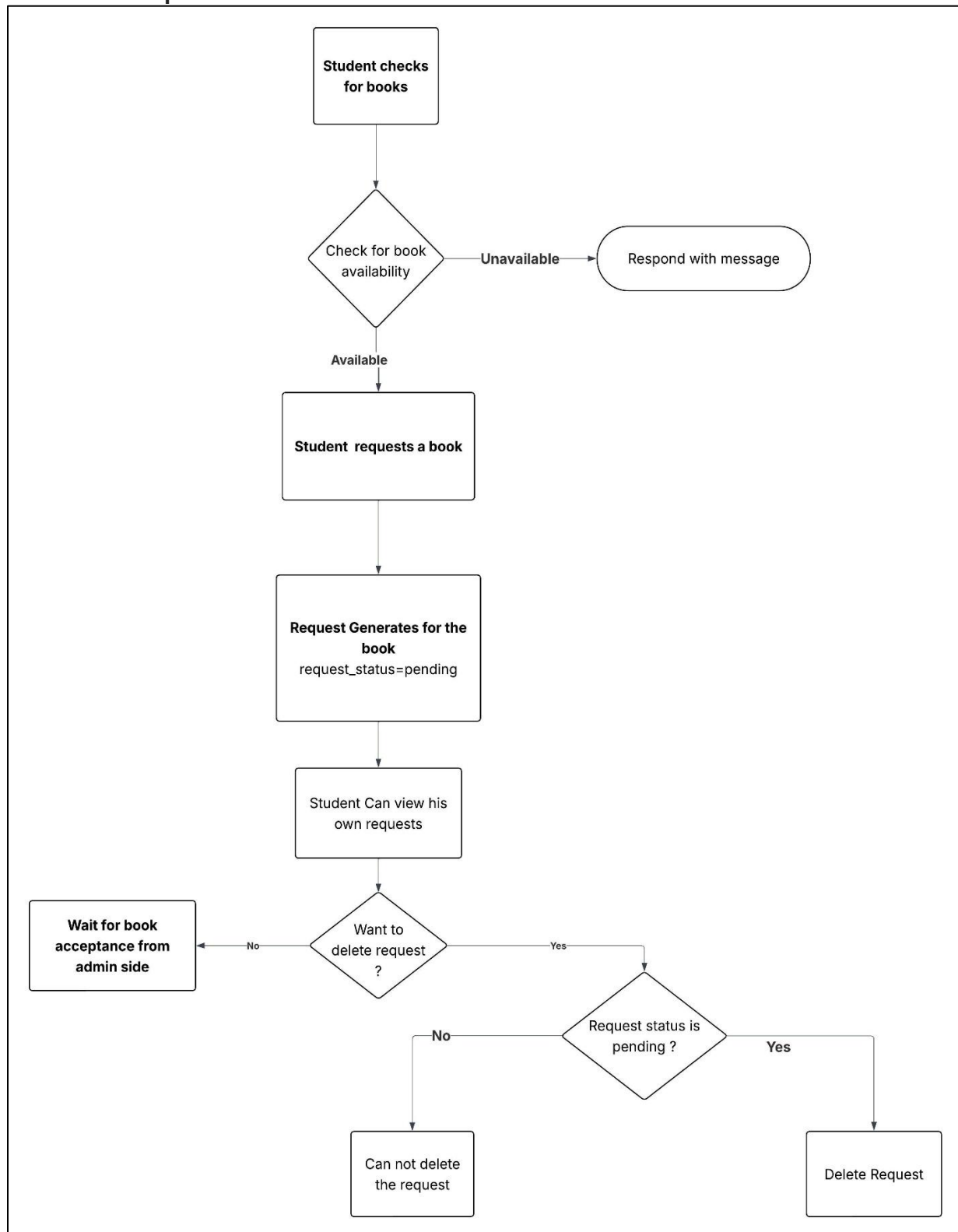
Method	Endpoint	Access	Description
POST	/roles	Admin	Create new role
GET	/roles	Admin	Get all roles
GET	/roles/:id	Admin	Get role by ID
PATCH	/roles/:id	Admin	Update role
DELETE	/roles/:id	Admin	Delete role

7. Flow Diagram

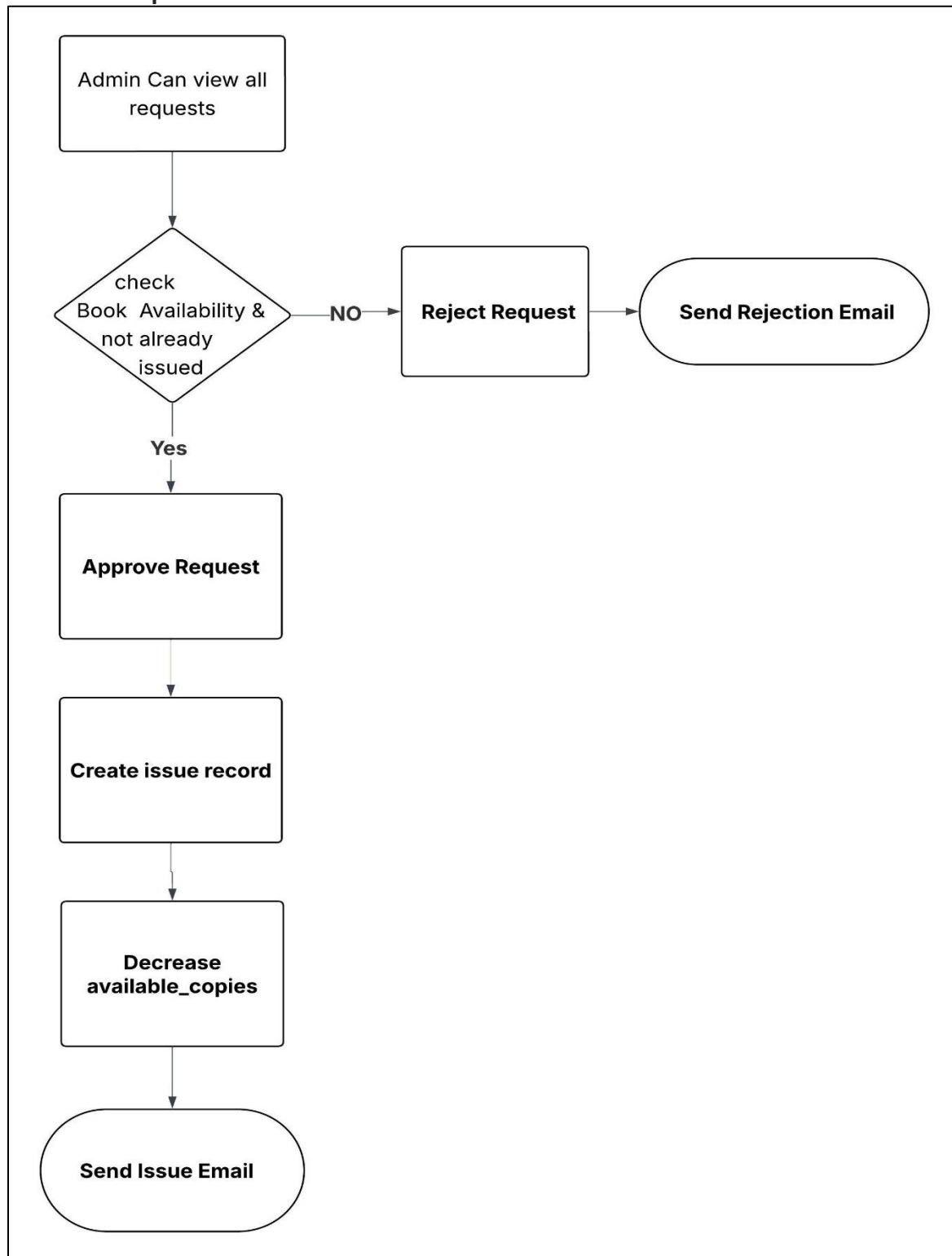
7.1 Registration flow



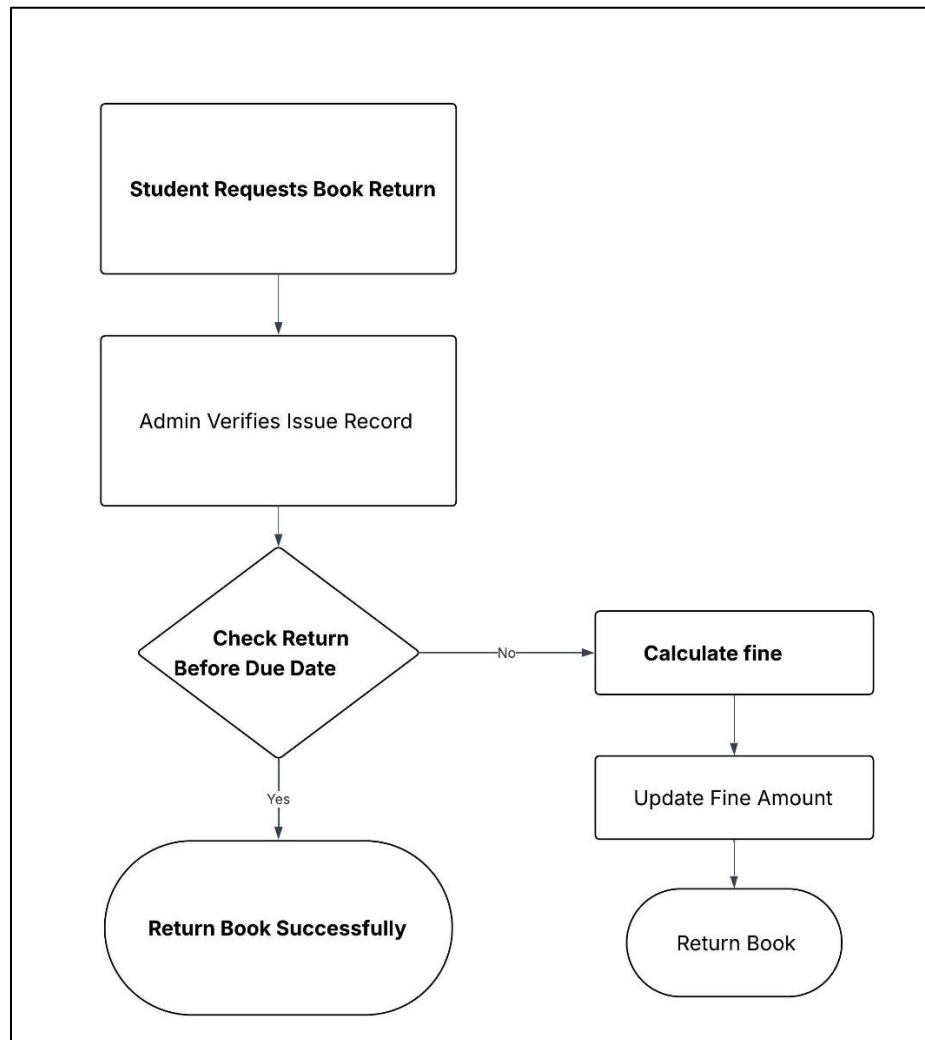
7.2 Student request flow



7.3 Book Request & Issue flow



7.4 Book Return Flow



8. System Architecture

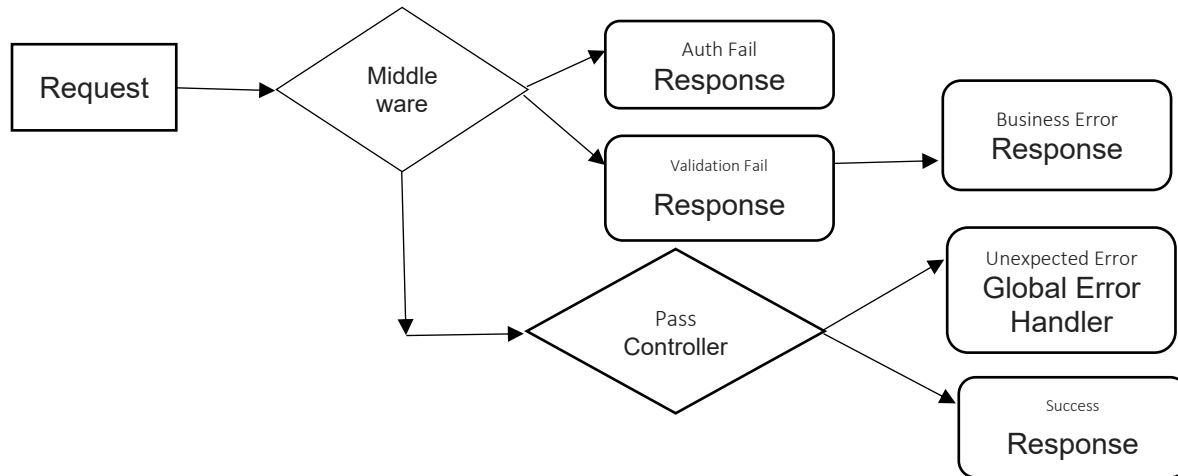
Layer Responsibilities

Layer	Responsibility
Client Layer	Sends HTTP requests (Postman)
Express Server	Parses request body, cookies, and matches routes
Middleware Layer	Authenticates user (JWT), authorizes role, validates request schema
Business Logic	Controllers handle core logic, orchestrate services, and interact with DB
Service Layer	Encapsulates JWT, email dispatch, logging, token generation
Data Layer	MySQL database accessed via connection pool with parameterized queries
External Services	Gmail SMTP for transactional emails

9. Validation Strategy

The system employs a multi-layered validation approach to ensure data integrity at every stage of the request lifecycle.

Validation Pipeline



The validation strategy uses three layers:

Layer 1 — AJV Schema Validation: All requests are validated against JSON schemas (body, query, params) before reaching the controller. Uses AJV v8 with strict mode (additional Properties: false).

Layer 2 — Controller Whitelisting: PATCH endpoints enforce an ALLOWED_FIELDS list to block unexpected or malicious fields.

Layer 3 — Database Constraints: Enforced via UNIQUE, NOT NULL, FOREIGN KEY, and ENUM constraints as a final safety net.

10. Future Enhancements

The system is designed with extensibility in mind.

#	Enhancement	Description	Impact
1	Frontend Integration	Build a React.js frontend with role-based dashboards	Full-stack deployment
2	Book Cover Images	Add image upload support (Multer + cloud storage)	Better UI/UX
3	Profile Image Upload	Allow users to upload profile pictures to cloud	User personalization
4	Due Date Reminders	Automated cron job to send email reminders 1-2 days before due	Reduce overdue returns